Reviewer Pack — Minimal Rank of Tropicalized Affine Sheaves vs AC0 Parity Ci…

Entry 4752cd699a11 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 16:02:47 UTC

Minimal Rank of Tropicalized Affine Sheaves vs AC0 Parity Circuit Size

Entry ID: `4752cd699a11` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 16:02:47 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry over Affine Sheaves **Field B** (complexity object): Complexity Theory: AC^0 Parity Circuit Complexity

Statement:

> ['For any AC^0 parity circuit C with size n, the minimal rank of its associated tropicalized affine sheaf is at least $\Omega(\log(n))$.', 'This lower bound holds for all circuits computing PARITY on n inputs.', 'The rank is measured as the dimension of the smallest vector space containing the tropicalization of the sheaf.']

Rationale (proposer's reasoning):

> ['Tropical geometry has been successfully applied to analyze arithmetic circuits, suggesting a potential for its application in circuit complexity theory.', 'Affine sheaves provide a rich structure that can capture the complexity of computations in AC^0 circuits.', 'This conjecture aims to exploit this structure to derive lower bounds on the size of AC^0 parity circuits.']

Taxonomy category: `AC0_PARITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2488ed858592ef01`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> (prereg LLM failed:)

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry and affine sheaves in AC0 parity circuit complexity - AC0 parity circuits minimal rank tropicalization affine sheaves - lower bound Ω(log(n)) tropicalized affine sheaves AC⁰ parity circuits

Top relevant hits considered: - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/1207.1925v1] Introduction to tropical algebraic geometry - [http://arxiv.org/abs/1206.1925v1] Counting Algebraic Curves with Tropical Geometry - [http://arxiv.org/abs/2304.02770v2] Tight Correlation Bounds for Circuits Between AC0 and TC0 - [http://arxiv.org/abs/1406.3065v2] Lower Bounds for Tropical Circuits and Dynamic Programs - [http://arxiv.org/abs/0810.5165v2] Some computations about Kazhdan-Lusztig cells in affine Weyl groups of rank 2 - [http://arxiv.org/abs/2510.17487v1] Directional Search for Persistent Gravitational Waves: Results from the First Part of LIGO-Virgo-KAGRA's Fourth Observin - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \to \mu^+\mu^-$ decay from the combined analysis of CMS and LHCb data

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_ac0_circuit(n):
        # Generate a random AC⁰ parity circuit of size n
        return [random.choice([0, 1]) for _ in range(n)]

    def tropicalize(circuit):
        # Tropicalize the affine sheaf associated with the circuit
        # This is a placeholder function. Implement the actual tropicalization logic here.
        return sum(circuit)

    def rank(sheaf):
        # Compute the rank of the tropicalized sheaf
        # This is a placeholder function. Implement the actual rank computation logic here.
        return len(set(sheaf))

    n = random.randint(5, 40)  # Sweep n through at least 4 distinct sizes inside each trial
    circuit = generate_ac0_circuit(n)
    sheaf = tropicalize(circuit)
```

```
    measured_rank = rank(sheaf)
    lower_bound = math.log(n, 2)  # Ω(log(n))

    return {
        "metric_name": "Rank of Tropicalized Sheaf",
        "metric_value": measured_rank,
        "instances_tested": 1,
        "conjecture_holds": measured_rank >= lower_bound,
        "counterexample": "" if measured_rank >= lower_bound else f"Circuit size {n} with rank {me
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(re
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        counterexample = next(res["counterexample"] for res in results if res["counterexample"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_74c4e170.py", line 54, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_74c4e170.py", line 38, in run_trial
    measured_rank = rank(sheaf)
                    ^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_74c4e170.py", line 33, in rank
    return len(set(sheaf))
               ^^^^^^^^^^
TypeError: 'int' object is not iterable
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which prevents a definitive evaluation of the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15145
2	propose	ollama_remote	glm4:latest	0	0	12246
3	preregistration	?	?	0	0	15136
4	novelty	ollama_remote	glm4:latest	0	0	8315
5	novelty	ollama_remote	glm4:latest	0	0	9639
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11735
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6529
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11671
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7407
10	judge	ollama_remote	glm4:latest	0	0	27192

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 125017 ms total latency. Provider mix: {'ollama_remote': 9, '?': 1}

(full prompt+response transcripts available in `research/audit/4752cd699a11.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4752cd699a11.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/4752cd699a11.tar.gz` (if generated)